Spring REST using Spring Boot 3

1. Create a Spring Web Project using Maven

Aim

To create a Spring Boot Web application using Maven.

Tools Required

- Java 17 or above
- Spring Boot 3
- Maven
- Eclipse/IntelliJ IDEA
- Postman
- Web Browser

Procedure

1. Open Spring Initializr.
2. Select Maven Project.
3. Choose Java as the language.
4. Select Spring Boot 3.
5. Add the Spring Web dependency.
6. Generate and download the project.
7. Import the project into Eclipse/IntelliJ.
8. Build the project using Maven.
9. Run the application.

Result

A Spring Boot Web project was successfully created using Maven and the application started successfully.

2. Spring Core - Load Country from Spring Configuration XML

Aim

To load a Country bean from the Spring XML configuration file.

Procedure

1. Create a Country class with code and name properties.
2. Create a Spring XML configuration file.
3. Define a Country bean with values:
    - Code: IN
    - Name: India
4. Load the XML configuration using ApplicationContext.
5. Retrieve the Country bean.
6. Display the country details.

Output

Country Code : IN
Country Name : India

Result

The Country bean was successfully loaded from the Spring XML configuration.

3. Hello World RESTful Web Service

Aim

To create a RESTful Web Service that returns "Hello World!!".

URL

GET /hello

Controller

HelloController

Method

@GetMapping("/hello")
public String sayHello()

```
{
   return "Hello World!!";
}
```

Sample Request

http://localhost:8083/hello

Sample Response

Hello World!!

Result

The Hello World REST API was successfully implemented and tested using the browser and Postman.

4. REST – Country Web Service

Aim

To create a REST API that returns India's details.

URL

GET /country

Controller

CountryController

Method

```
@RequestMapping("/country")
public Country getCountryIndia()
```

Procedure

1. Load the India bean from the Spring XML configuration.

2. Return the Country object.
3. Spring Boot automatically converts the object into JSON.

Sample Response

```
{
   "code":"IN",
   "name":"India"
}
```

Result

The REST API successfully returned the country details in JSON format.

5. REST – Get Country Based on Country Code

Aim

To retrieve country details based on the country code.

URL

GET /countries/{code}

Controller Method

```
@GetMapping("/countries/{code}")
public Country getCountry(@PathVariable String code)
```

Procedure

1. Read the country code using @PathVariable.
2. Load all countries from country.xml.
3. Search the country list.
4. Match the country code without considering case.
5. Return the matching country.

Sample Request

http://localhost:8083/countries/in

Sample Response


```
{
   "code":"IN",
   "name":"India"
}
```


Result

The REST service successfully returned the requested country's information.

6. Create Authentication Service that Returns JWT

Aim

To implement JWT Authentication using Spring Security.

Tools Required

- Spring Boot 3
- Spring Security
- JWT (JJWT Library)
- Maven
- Postman

Procedure

1. Add Spring Security dependency.
2. Add JJWT dependency.
3. Create an AuthenticationController.
4. Create the endpoint `/authenticate`.
5. Read the Authorization header.
6. Decode the username and password.
7. Generate the JWT token.
8. Return the generated token.
9. Test the API using Postman or Curl.

URL

GET /authenticate

Sample Request

curl -u user:pwd http://localhost:8090/authenticate

Sample Response

{

"token":"eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNTcwMzc5NDc0LCJleHAiOjE1NzAzODA2NzR9.t3LRvlCV-hwKfoqZYlaVQqEUiBloWcWn0ft3tgv0dL0"
}

Advantages of JWT

- Stateless authentication
- Secure token-based authentication
- Reduces server-side session storage
- Faster authentication
- Suitable for REST APIs

Result

JWT Authentication was successfully implemented, and the authentication service generated a valid JWT token after successful login.